

SQL:

Structured query language.

Designed

to

manage

data stored in relection of databases,

"SELECT * FROM colobs

1. CREATE TABLE dileep (id INTEGER, name
Text, age INTEGER);

INSERT INTO

Jileep (id,

name

1

age)

VALUES

(' , ' Dileep', 24);

4

SELECT

name FROM celebs;

}

UPDATE
Celebs

SET age = 22

CREATE
TABLE

INSERT INTO

SELECT quenes data
fortal

WHERE Ed
d = 1;
if a 'clause'

2

UP DATE

SELECT * from celebs;

ALTER TABLE

DELETE

FROM;

Dad a

www

column to table,

ALTER TABLE celebs ADD COLUMN

SELECT Fimm colebs;

UPPATE Cobeles

SET

dilcop coluirm l

@401

dreep'

WHERE id = !;

dileep, olumn
TEXT;

UPDATTE

Celebs

SET batter_____ handle -

"@ddeep'

WHERE id = 1;

DELETE FROM Celebs WHERE twitter handle "Is
'@dbep';

```
SELECT * FROM celebs;
```

```
SELECT name, imdb. rating from movies;
```

```
SELECT DISTINCT genre
FROM movies
return
unique valleees .
```

⌘

(81)
NOLL

```
SELECT * FROM Movies
WHERE imdb_rating >8;
```

مرا)



= 8;)

SELECT

name, Imabareting FROM movies

WHERE indberetning >8;

SELECT

* FROM movies

WHERE

name

LIKE Sezen';

Seven'; C.

wild card character)

T

(Special
Operatie)

SELECT * FROM movied

WHERE name LIKE 'ay.';

SELECT

#fram mouth

WHERE

name

LIKE "+/% mang.';

11.

wild card charactel

A.1.

mather

names that begin with "A"

matches

all moves that and with 'a '

SELECT * FROM movies

WHERE

name BETWEEN

'A' AND 'J';

operator.

Select

Me from movies

where year between 1990 and 2000;

D

SELECT From movies

WHERE

Year BETWEEN

1980 AND 2000 AND genre =

comedy'

;

SELECT * FROM

Movics & WHERE

genre =

Comedy' OR

year < 1980;

↓

SELECT * FROM movies

ORDER BY imdb_rating DESC

Descending order.

Select * from movies where year between 1990 and 2000

1990 and 2000 ORDER BY

year

DESC ;

SELECT

* FROM

movies

ORDER BY

indbareting ASC

LIMIT 31

I

SELECT

clause

WHERE

LIKE

/

clause

BETWEEN - spend

operator

Special

AND COR

ORDER BY

LIMIT

(ASC / DESC)

(МОЖНО)

SELECT

COWNI(1) FROM Jake apps: WHERE
price = 0;

SELECT price, COUNT (H) FROM
take-apps

GROUP BY price;

SELECT

price ,
COUNT (x), downloads FROM
fake-cupps

WHERE downloads> 20000

GROUP BY

price;
↑

SELECT Sum (downloads) FROM fakenapps;

```
SELECT
    MAX Colan leads
    FROM Jake - apps';
!
```

```
SELECT
    sum (dawnloads) From
    fake_apps
    Category
```

Group By
category;

```
SELECT
    name, category, MAX (downloads)
    from Jake_apps
```

Group By
Category:

```
SELECT
    Дне
    Avg (downloads) from fake apps;
```

```
SELECT
    name, price
    1
```

ROUND (AVG (downloads), 2) FROM
Jake__apps

GROUP BY price.

SELECT

price, ROUND (AVG (downloads))

FROM Jake Apps

Group By

Price

Aggregate functions

I
↑

COUNT

GROUP BY

SUM)

MAX()

MIN

AVG()

ROUND() .

Multiple tables:

```
CREATE TABLE artists (id INTEGER PRIMARY KEY, name  
TEXT  
TEXT);
```

primary key :

unique identifier

J

None of the

values in this col and NULL

→

Each valice

in this

Column is unique

```
"SELECT albums.name, albums. year,  
artists, name
```

```
FROM albums, artists;
```

```
SELECT * FROM albums
```

```
JOIN artists ON
```

```
albums. artist_id =
```

```
artists.ed
```

```
;
```

SELECT

* from albums

LEFT

JOIN

artists ON

albums, artist_id =
artists.id;

Null values

will be

considered.
)

SELECT

albums.name As

|
Album',

rename

a column

FROM

albums.year,

artists.name As 'Artists'

albums

JOIN artists ON

albums.artist_id = artists.id

WHERE

albums. year > 1980;

↑
1

primary key

Foreign key

Join

INNER Join

LEFT

OUTER JOIN

As,

INNER JOIN!

2 Tables

SELECT

COMM

IMUNITY__NAME, ID

FROM

LTV CONNECTION DAFTA

В тольив

```
INNER JOIN  
LT_COMMUNITIES
```

```
ON LT_CONNECTION_DATA. R?:DD =  
LT_COMMUNITIES. ID
```

```
WHERE DD = 30;
```

```
CM. COMMUNITY-NAME,  
COM.DP_ADDR
```

```
SELECT
```

```
CM.
```

```
FROM
```

```
LTX CONNECTION BAJA CON
```

```
INNER JOIN LT_CommunaY-TEMS CR
```

```
ON CON. RD_ID = CLORPRD
```

Porta Jon

LT_CommoNITIES (M)

ON Cf.comMUNITY_ID=CMADD

INFLERE DO

-ADDDR =

· 168 · 127.42.225;

L

PD

SQL

CREAT TABLE

employed

C

emp_id INT (PRIMARY
KEY,

first_name (VARCHAR
(40),

CREATE TABLE client (

INT

DB 1

client -id

client- name

PRIMARY KEY VARCHAR(40),

INT,

brand id

FOREIGN KEY (branch_id)
REFERENCES

last-name

VARCHAR (40),

branch (branch-id) ON DELETE SET
NULL.

Birth-day

DATE

Sex

VARCHAR
(1) ,

INT,

Salary INT

Super_id

branch_id INT

CREAT TABLE branch I

branch_id

INT

PRIMARY KEY,

Branch- mame

VARCHAR
(40) ,

mgr_id

INT,

mgr_start_date DATE,

CREATE TABLE works with (

emp_id INT

client_id INT,

total_sales INT,

PRIMARY KEY (emp_id, client_id)

FOREIGN KEY (emp_id) REFERENCES

employee (emp_id) ON DELETE

CASCADE, FOREIGN KEY (client_id)

REFERENCES Ment (client_id) ON

DELETE SET CASCADE

FOREIGN KEY (mgr_id) REFERENCES employee (emp_id) ON
DELETE SET NULL

ALTER TABLE

employee

ADD FOREIGN KEY

(branch_id)

REFERENCES brand (branch_id)

ON DELETE SET NULL

ALTER TABLE

employee

ADD

FOREIGN KEY

(Super_id)

REFERENCES employee
(emp_id)

ON

DELETE SET NULL;

";

CREATE

TABLE

branch -

Supplier (

branch_id INT,

Supplier_name

e

VARCHAR (40),

Supplier_name VARCHAR (46),

PRIMARY KEY (branch_id,

supplier_name

FOREIGN KEY (branch_id) REFERENCES branch (branch_id) ON DELETE
SET CASCADE

);

Corporante

INSERT INTO employee VALUES (100, 'David', 'wallace',
'1967-11-17', 'M', 205000,

INSERT

INTO

NULL, NULL) ;

```
branch VALUES (1, 'Corporate', ID,  
'2006-02-09');
```

```
UPDATE employee
```

```
SET  
branch_id=1
```

```
WHERE emp_id = 100;
```

```
INSERT INTO
```

```
employee VALUES (101, 'Jan', 'Levinson',  
'1961-05-11', 'F', 1000, 100, 1),
```

```
Scranton
```

```
INSERT INTO
```

```
строке
```

```
VALUES  
(109,
```

```
109, 'Michel', 'Croft',
```

```
('1984-03-15', 'M',  
75000,
```

```
100, NULL);
```

```
INSERT INTO
```

```
branch
```

```
VALUES (2, 'Scranton', 102,  
'1982-04-06');
```

```
UPDATE employee
```

```
SET branch_id = 2
```

```
WHERE
```

emp_id
102;

→ SELECT * FROM employee;

T
SELECT *

from employee ORDER
BY

Salary
ASC/DESC;

→
SELECT

→>>
SELECT

*
*

FROM

emplyce

ORDER BY

SEY, name;

from emplyce

LIMIT 5;

کا کیا گیا
SELECT

first_name, emplyce,
last_name

FROM eupbyce;

SELECT

first name As forename

, employer . Last name As Surrname

from emplace;

SELECT

DISTINCT

sex

FROM employee;

SELECT * FROM

employee

WHERE Gex='M';

SELECT

emp-id, first_name, Last_name FROM
employee

WHERE

birthday >= 1970-01-01

SELECT * employee WHERE

SELECT

FROM employee

branch_id = 2 AND

Da

Sex = 'f' ;

WHERE (birth day >=
'1970-01-01' AND Sex="F")

OR Salary > 80000;

SELECT *

FROM

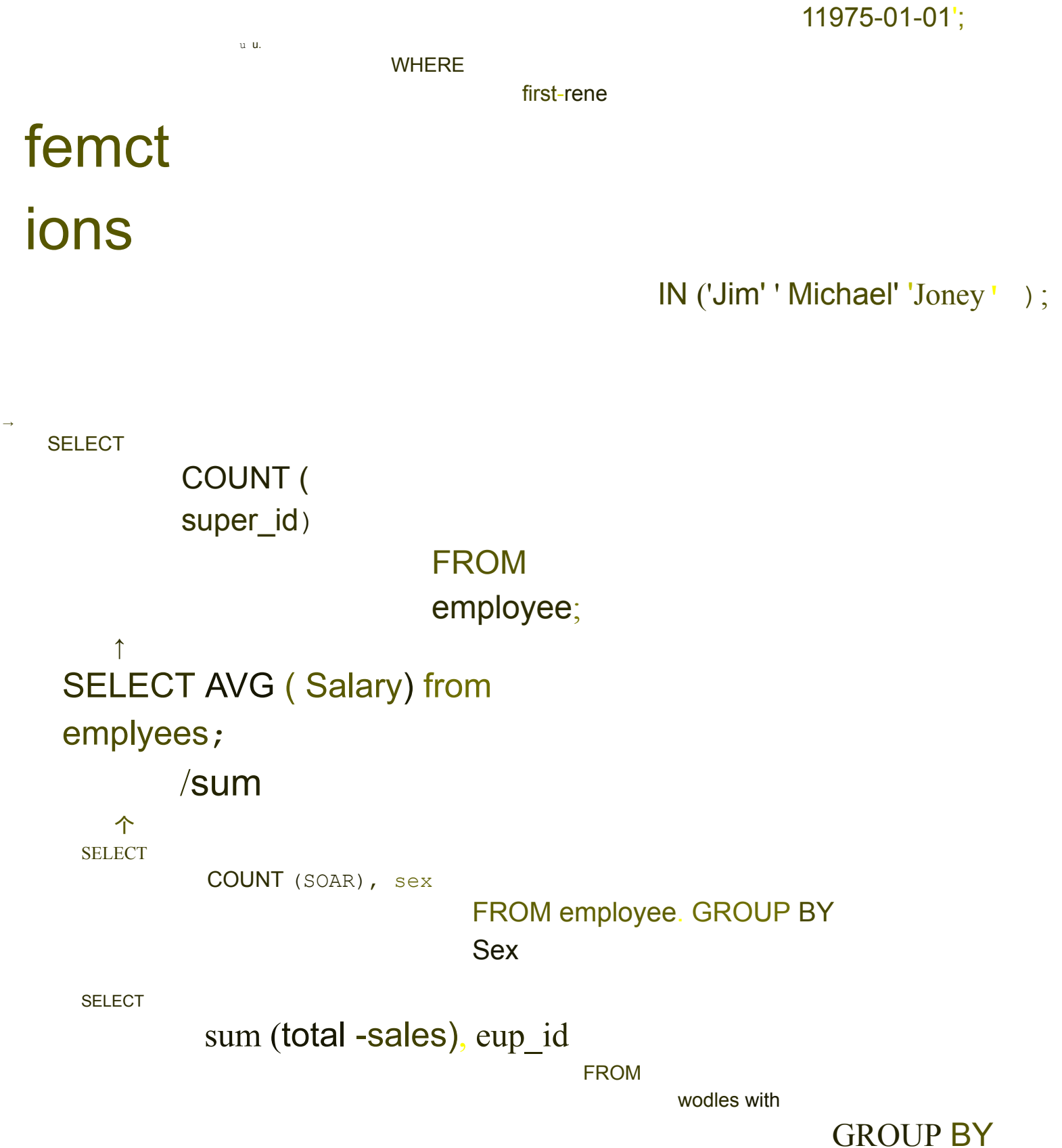
employee

WHERE

birth-day
BETWEEN

1970-01-01' AND

functions



client_id;

SELECT

SUM (total-Sales), client_id FROM works with

GROUP BY
client-id

wildcards:

SELECT

From client where client_name LIKE '%LLC ' s,

'% Label ". " ;

10%';

Union!

1

% Highschool%;

SELECT employee,
first-name

AS Employee - Branch - Names

FROM employee

UNION

SELECT brand.brand name

FROM

brand;

->

SELECT

AS

client . clint

Branch from client.

AS

Non-employee entities, clinta branch id

UNION

SELECT brach_suppli

Suppl

tranh

-Suppphin bronhid

From

brach - Sryption;

↑

employee emp-id, employee first-remu,

brank, brand-name

SELECT

from emphyre

JOIN

banh

ON

emphyse. empaid | borach .
mgr_id;

یا

LEFT JOIN | RIGHT
JOIN.

Nested

queries

SELECT

emphyce · find_name, emplyce

& Last

FR

From

emphyce

WHERE

employee emp_id IN (SELECT works_
with.emp_id

дегест

with

From

Works

WHERE

woles with- total_sales >
5000);

1

USE

Databore

-home

SHOW

Daterboness;

DB 3

Data: Data in Simple words, **is** distinct pieces of
information can's collection of

facts related to any entity.

Data base: A container of
dote,

container of data collection

Data base management

System:

of related data

data

collection of programmes
which enables.

users to access
database,

manipulate data, and

(

A technology to
store

with

represent data.

and retrieve data with utmost
efficiency along

appropriate security
measures .

Types of DBMS

I

Hierarchical

Network

Relational object - oriented.

1) Hitical DAMS: "parent-child relationship of storing data, a tree structure

with nodes
representing

reccords jex:
"windows

registry

2

مل

Network DBMS :

Supports many

to

many' redations, usually results
in

complex database structure.

ex:

RDM Server

32

*
.

RDBMS!

Database relationships in form of tables, also known as relations,

it

usually have

pre-defined datatypes, most popular
ones

Ex.

Microsoft SQL

Server

Database .

ODBMS :

Supports storage of
new

data types, the

Data to be stored

in the form of objects.

Structured Query language
(SQL);

A standard programming
language which is

With

-

used for managing relations

database

delete rows of data,

SQL,

you can

modify databases, add, update

or

retrieve

Sulosets of information from a

delabar and

A

any more

Relectiont

debabane like my SQL Databases, Oracle, MS SQL
sybeerecte

uses SCOL

My SQL work bench?

Visual Database designing and modelling
of defobare

port

3306

schema: ?

SQL command
categoricó (4)

DDL

HDML

- DCL

- Ada potinition language

- Data Manipulation Language

+

↑

access tool.

SQL Control Language

SQL

Transaction Control language

DAL:

commands

are

used to define database schema

Create,

Drop, Alter, Truncate,

Comment, Rename

DML :

Commands that deal with the manipulation of

data

present. present
in data

SELECT
, INSERT, UPDATE, DELETE

DCL

:

commands

controls of

which mainly deals with the rights

database System

GRANT, INVOKE

& permissions
and other

たし

commands **aluch** mainly deal with the

transaction &

data have

COMMIT, ROLL BACK, SAVEPOINT, SET TRANSACTION.

Data making

DB

how

do we

design

diabare '

Complete

banlar

Normalive

using

Entity - Relationship Diagram. (Ex diagram)